📘 Guide d'intégration API — UNOVAPP Backend

Contrat API pour l'équipe Frontend Android. Mis à jour le **2026-06-27** (migration Render → **VPS Hostinger**, passerelle unique). 6 services : auth/user/social (NestJS), video/notification (Go), transcoding (Python).

🌍 URL de base — passerelle unique

`http://152.239.118.90/api/v1`

Domaine	Chemins	Service
Authentification	/api/v1/auth/*	auth
Profils, follow, avatar	/api/v1/users/*	user
Likes/commentaires/vues/recherche	/api/v1/videos/:id/like·/comments·/view, /api/v1/search	social
Vidéos, feed	/api/v1/feed, /api/v1/videos/*	video

✅ Une seule base URL, le serveur route en interne (plus de « double hôte »). ✅ Toujours allumé (plus de veille). ⚠️ HTTP (pas encore HTTPS) → sur Android, autoriser le cleartext vers cette IP (network_security_config.xml). ℹ️ Transcoding & Notification = internes, non exposés.

🔑 Convention

Header : Authorization: Bearer <accessToken>. accessToken 1 h, refreshToken 7 j (rotation). Sur 401 → POST /auth/refresh { refreshToken }, sinon login. Même JWT_SECRET partagé (payload { sub, email }, pas de username). Pagination cursor

(feed/commentaires) ou page (users).

🔐 AUTH — /api/v1/auth/*

POST /auth/register (Non auth) — body { email, username(3-20), password(≥8), phone_number? } → { message }, OTP email 6 chiffres (15 min). Ne crée pas encore le compte.

POST /auth/verify-email (Non) — { email, otp_code(6) } → { accessToken, refreshToken } (compte créé). Erreurs 400/409.

POST /auth/login (Non, 5/min) — { email, password } → tokens. 401/429.

POST /auth/refresh (Non) — { refreshToken } → nouveaux tokens (rotation). 401.

POST /auth/logout (Auth) — révoque l'accessToken (Redis) → 204.

POST /auth/me (Auth) → { userId, email }.

POST /auth/send-otp (Non, 5/min) — { phone_number } → OTP SMS (Infobip, 5 min).

POST /auth/verify-otp (Auth) — { phone_number, code } → { verified:true }.

POST /auth/forgot-password (Non, 3/min) — { email } → réponse générique, code OTP email 15 min.

POST /auth/reset-password (Non, 5/min) — { email, otp_code(6), newPassword(≥8) } → { message }. Code annulé après 5 essais. Invalide les sessions.

PATCH /auth/change-password (Auth) — { currentPassword, newPassword, confirmPassword }.

POST /auth/google (Non) — { id_token } → { accessToken, refreshToken, user, isNewUser }.

👤 USER — /api/v1/users/*

GET /users/me (Auth) → profil privé complet (email, phone, wallet, stats).

POST /users/me/avatar/presign (Auth) — { contentType } → { key, uploadUrl, publicUrl, ... } ; puis PUT du fichier sur uploadUrl ; puis confirmer.

PUT /users/me/avatar (Auth) — { key } → profil avec avatar_url.

GET /users/search?q=&page=&limit= (Non) — q min 2 car → { data, total, page, limit }.

GET /users/:id (Bearer optionnel → is_following) → profil public.

PATCH /users/:id (Auth, son propre UUID) — { display_name?, bio?, username?, avatar_s3_key? }. ⚠️ Pas de PATCH /users/me.

POST/DELETE /users/:id/follow (Auth) — follow idempotent / unfollow.

GET /users/:id/followers · /following (?page=&limit=) → listes paginées.

🎬 SOCIAL — /api/v1/videos/:id/{like,comments,view} + /search

POST /videos/:id/like (Auth) → { liked, likes_count } (toggle). ⚠️ likes_count en STRING ici.

POST /videos/:id/comments (Auth) — { content(1-500), parent_id? } → entité commentaire.

GET /videos/:id/comments?cursor=&limit= (Non) → { data, next_cursor, has_more }.

DELETE /videos/:id/comments/:commentId (Auth, auteur) → 204.

POST /videos/:id/view (Non) → { message }.

GET /search?q=&cursor= (Non) → { videos, users }.

📹 VIDEO — /api/v1/feed + /api/v1/videos/*

GET /feed?cursor=&limit= (Bearer optionnel → is_liked) → { data:[...], next_cursor, has_more }.

POST /videos/upload (Auth) — headers Upload-Length (max 500 Mo), Upload-Metadata → { upload_id, video_id, upload_url, expires_at } + headers TUS.

HEAD /videos/upload/:uploadID (Non) → headers Upload-Offset/Length (reprise).

PATCH /videos/upload/:uploadID (Auth) — headers Tus-Resumable: 1.0.0, Content-Type: application/offset+octet-stream, Upload-Offset ; body chunk binaire (max 512 Ko) → 204. À la fin → event video.uploaded (transcodage).

GET /videos/:id (Non) → métadonnées (status, renditions 144p/240p/480p/720p, hls_manifest_url, thumbnail_url, compteurs).

GET /videos/:id/manifest (Non) → 302 vers le HLS si prêt, 202 si en cours, 404 sinon.

DELETE /videos/:id (Auth, créateur) → 204.

🎞️🔔 TRANSCODING & NOTIFICATION — internes

Consumers RabbitMQ : rien à appeler. Transcodage → 4 renditions HLS + thumbnail ; suivre via GET /api/v1/videos/:id (status processing→published). Push FCM auto : data.type = video_ready / video_liked / video_commented / new_follower. Abonnement

Android : subscribeToTopic("user-" + userId). Push en simulation tant que FCM_PROJECT_ID vide.

🚦 Statut (2026-06-27 — VPS Hostinger)

Ubuntu 24.04, 2 vCPU / 8 Go, Frankfurt. 6 conteneurs Docker derrière Nginx. PostgreSQL Supabase, Redis Upstash, RabbitMQ CloudAMQP, S3 Supabase. ✅ Parcours auth validé end-to-end (register → OTP email Brevo → verify → login → profil).

Dépendance	Provider	État
Base de données	Supabase	🟢 Actif
Email OTP / reset	Brevo	🟢 Actif (⚠️ spams)
Stockage	Supabase S3	🟢 Actif
Cache/sessions	Upstash Redis	🟢 Actif
Messages	CloudAMQP	🟢 Actif
SMS	Infobip	🟠 à confirmer
Push	FCM	🟠 simulation

📋 Codes d'erreur

Code	Sens	Action
400	Validation	Erreurs par champ
401	Token KO	refresh sinon login
403	Interdit	Ne pas réessayer
404	Introuvable	État vide

Code	Sens	Action
409	Conflit (déjà pris)	Afficher message
412/415	Headers TUS	Corriger
413	Trop volumineux	Redécouper
429	Rate limit	Attendre
503	Service externe KO	Réessayer